

SERVICENOW PROJECT DOCUMENTATION

Script-Controlled ACL

Restrict Record Access Based on Field Value

Project Information	Details
Project Type	ServiceNow Security / Access Control
Primary Artifact	Script-Controlled Access Control List (ACL)
Target Table	Institution Details — u_institution_details
Primary Operation	Read
Business Condition	Branch = EEE
Role Used for Read Access	bb1
Administrator Access	admin
Additional ACL Operations	Create, Write, Delete
Reference Structure	ServiceNow implementation documentation format

Prepared as a project implementation and presentation document

1. Executive Summary

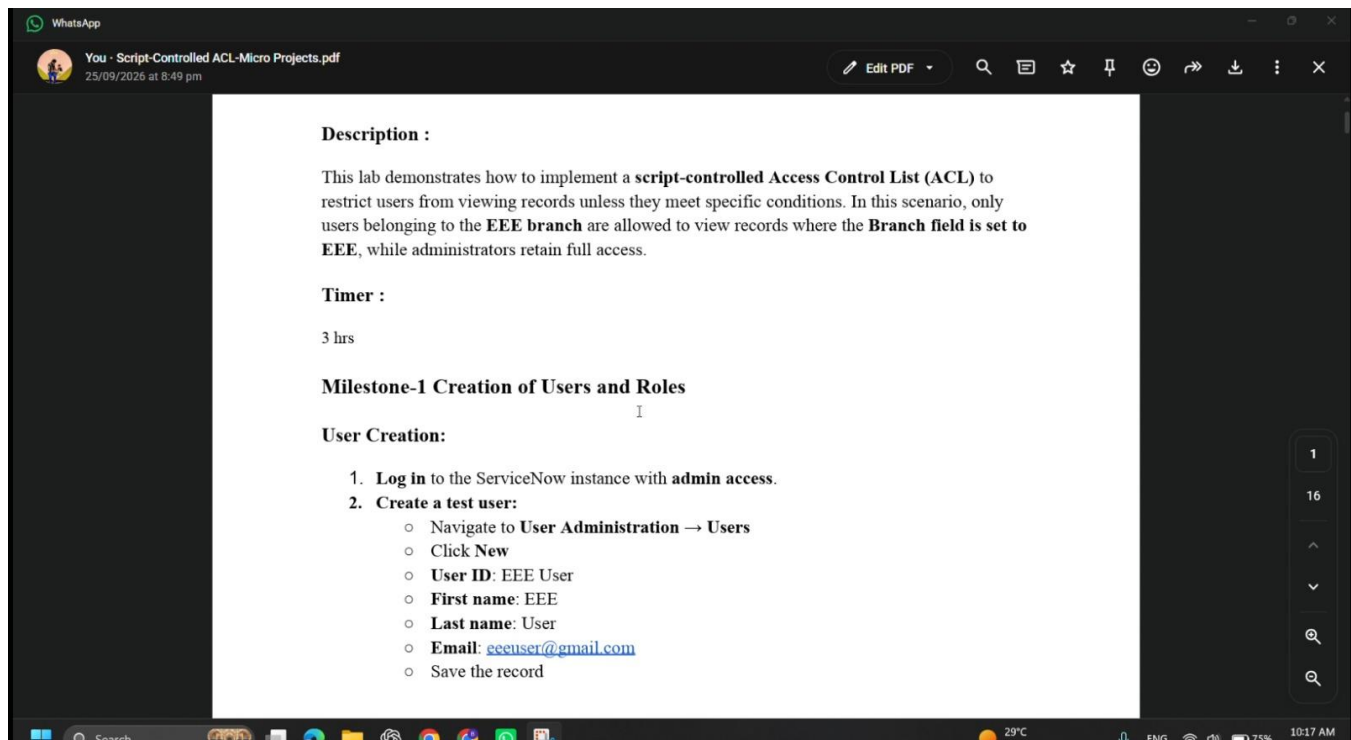
This project implements record-level security in ServiceNow using Access Control Lists (ACLs), custom roles, a custom table, a branch-based data condition, and a server-side ACL script. The project is designed around an Institution Details table containing student and faculty information, where records are created for multiple branches such as ECE, EEE, and CSE.

The central security requirement is that EEE users should be able to view EEE records, while administrators retain full access. The Read ACL combines the Branch = EEE data condition with role/script evaluation. The project then extends the same table with Create, Write, and Delete ACLs using separate roles, demonstrating operation-specific access control.

The supplied project material defines the ACL script as an authorization gate: administrators return true, users with the bb1 role return true, and all other users return false. The supplied configuration also specifies Branch = EEE as the Read ACL data condition. [filecite2turn0file12L65-L832](#)

Project Outcome

- Create a controlled test user and custom roles.
- Create the u_institution_details custom table.
- Store records with ECE, EEE, and CSE branch values.
- Create a Read ACL with the EEE branch condition and bb1 role.
- Use a script to allow administrators and authorized bb1 users while denying others.
- Create separate Create, Write, and Delete ACLs using bb2, bb3, and bb4 respectively.
- Validate access by impersonating different users.



2. Project Overview & Scope

The project demonstrates how ServiceNow ACLs can protect records at the table and operation level. The implementation covers the four core record operations: Read, Create, Write, and Delete.

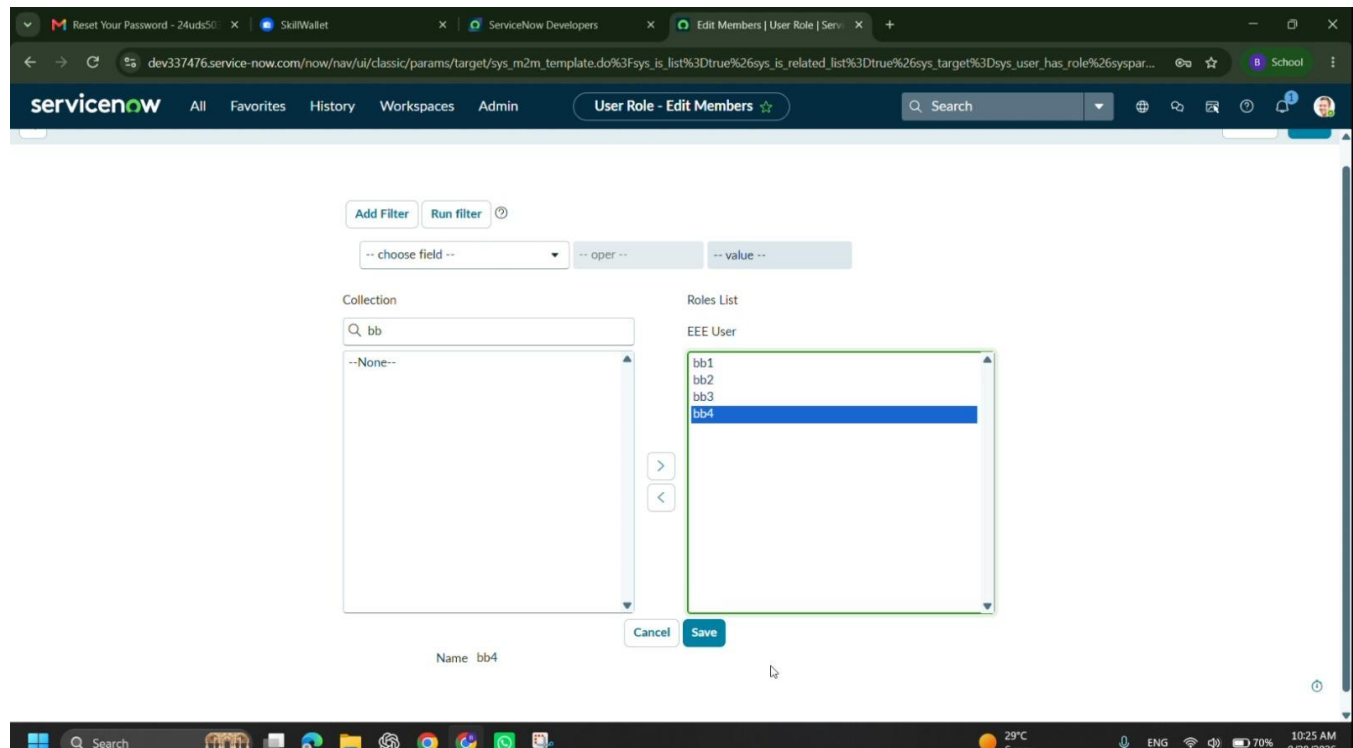
Milestone	Artifact	Purpose	Trigger / Scope
1	User & Role Setup	Create test user and custom roles	User Administration
2	Custom Table	Create Institution Details table	System Definition > Tables
3	Read ACL	Restrict visibility to EEE records	Read operation
4	Read Verification	Test bb1, non-bb1, and admin access	Impersonation
5	Create ACL	Control record creation	Create operation
6	Write ACL	Control record modification	Write operation
7	Delete ACL	Control record deletion	Delete operation
8	End-to-End Validation	Confirm security behavior	Forms / Lists

Scope

- ServiceNow Personal Developer Instance or equivalent development instance.
- Security configuration performed with administrator privileges.
- Security_admin elevation is required when creating or modifying ACLs.
- Project table: u_institution_details.
- Branch values used in the source project: ECE, EEE, CSE.

Out of Scope

- Application-specific business workflows beyond ACL enforcement.
- Client Scripts or UI Policies.
- External identity-provider integration.
- Production deployment or change-management approval.



3. Security Architecture

The implementation separates identity, role membership, record data, and operation permissions. A user is evaluated against the required role and ACL logic, while the Read ACL also uses the Branch = EEE data condition defined in the project.

Layer	Component	Responsibility
Identity	ServiceNow User	Represents the person attempting the operation.
Authorization	bb1 / bb2 / bb3 / bb4 roles	Provides operation-specific privileges.
Data	u_institution_details	Stores student, faculty, branch, email, phone, and description data.
Record condition	Branch = EEE	Restricts the Read ACL to EEE branch records.
Logic	ACL script	Returns true for admin or bb1 and false for all other users in the supplied Read ACL script.
Operations	Read / Create / Write / Delete	Defines what type of action is being protected.

High-Level Access Flow

1. User attempts to access or modify a record.
2. ServiceNow evaluates the ACL for the requested operation.
3. Required role and ACL conditions are evaluated.
4. For the Read ACL, the project uses Branch = EEE as the data condition.
5. The supplied script returns true for admin users, true for users with bb1, and false for all others.
6. ServiceNow grants or denies the requested operation based on the ACL evaluation

User ID	Name	Email	Active	Created	Updated
EEE User	EEE User	eeuser@gmail.com	true	2026-09-27 21:49:38	2026-09-27 21:49:38
efren.baucher	Efren Baucher	efren.baucher@example.com	true	2012-02-17 19:04:53	2026-09-22 20:33:14
elaine.renzi	Elaine Renzi	elaine.renzi@example.com	true	2012-02-17 19:04:52	2026-09-22 20:33:12
eldon.sutch	Eldon Sutch	eldon.sutch@example.com	true	2012-02-17 19:04:53	2026-09-22 20:33:22
eli.bettner	Eli Bettner	eli.bettner@example.com	true	2012-02-17 19:04:51	2026-09-22 20:33:15
elisa.gracely	Elisa Gracely	elisa.gracely@example.com	true	2012-02-17 19:04:52	2026-09-22 20:33:12
eliseo.wice	Eliseo Wice	eliseo.wice@example.com	true	2012-02-17 19:04:53	2026-09-22 20:33:13
ella.pahnke	Ella Pahnke	ella.pahnke@example.com	true	2012-02-17 19:04:49	2026-09-22 20:33:19
elmo.dagenais	Elmo Dagenais	elmo.dagenais@example.com	true	2012-02-17 19:04:53	2026-09-22 20:33:20
elmo.gabouer	Elmo Gabouer	elmo.gabouer@example.com	true	2012-02-17 19:04:53	2026-09-22 20:33:11
elvira.blumenthal	Elvira Blumenthal	elvira.blumenthal@example.com	true	2012-02-17 19:04:52	2026-09-22 20:33:18
emery.reek	Emery Reek	emery.reek@example.com	true	2012-02-17 19:04:49	2026-09-22 20:33:13
emilia.oxley	Emilia Oxley	emilia.oxley@example.com	true	2012-02-17 19:04:49	2026-09-22 20:33:19
emilio.lampkin	Emilio Lampkin	emilio.lampkin@example.com	true	2012-02-17 19:04:49	2026-09-22 20:33:22
enrique.oroark	Enrique Oroark	enrique.oroark@example.com	true	2012-02-17 19:04:50	2026-09-22 20:33:13
eric.schroeder	Eric Schroeder	eric.schroeder@example.com	true	2007-07-03 11:50:20	2026-09-22 20:33:17

4. System Prerequisites & Environment Setup

Requirement	Configuration
Platform	ServiceNow development / Personal Developer Instance
Administrative access	admin or equivalent configuration privileges
Security role	security_admin elevation for ACL configuration
Main navigation	User Administration, System Definition, System Security
Target table	u_institution_details
Test branches	ECE, EEE, CSE
Test roles	bb1, bb2, bb3, bb4

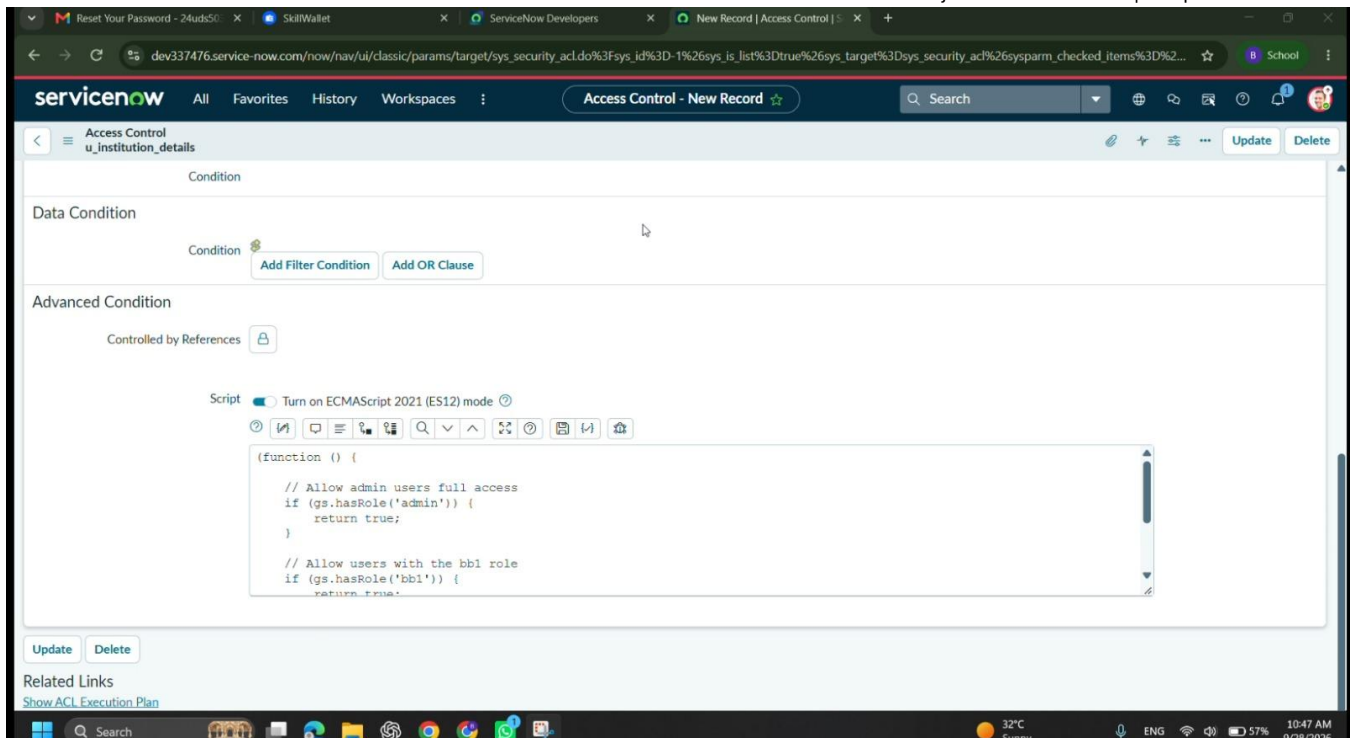
Important Navigation Paths

- User Administration → Users
- User Administration → Roles
- System Definition → Tables

The screenshot displays the ServiceNow interface for configuring the 'u_institution_details' table. The table structure is as follows:

Column label	Type	Reference	Max length	Default value	Display
Updated by	String	(empty)	40	40	false
Updates	Integer	(empty)	40	40	false
Updated	Date/Time	(empty)	40	40	false
Sys ID	Sys ID (GUID)	(empty)	32	40	false
Created by	String	(empty)	40	40	false
Created	Date/Time	(empty)	40	40	false
Student Roll Number	String				false
Student Name	Reference	u_			false
Faculty Name					false
Branch					false
Email					false
Phone Number					false
Description					false

Buttons at the bottom include 'Delete', 'Update', and 'Delete All Records'. The 'Student Name' column is currently being edited, showing a search box with the text 'u_'.

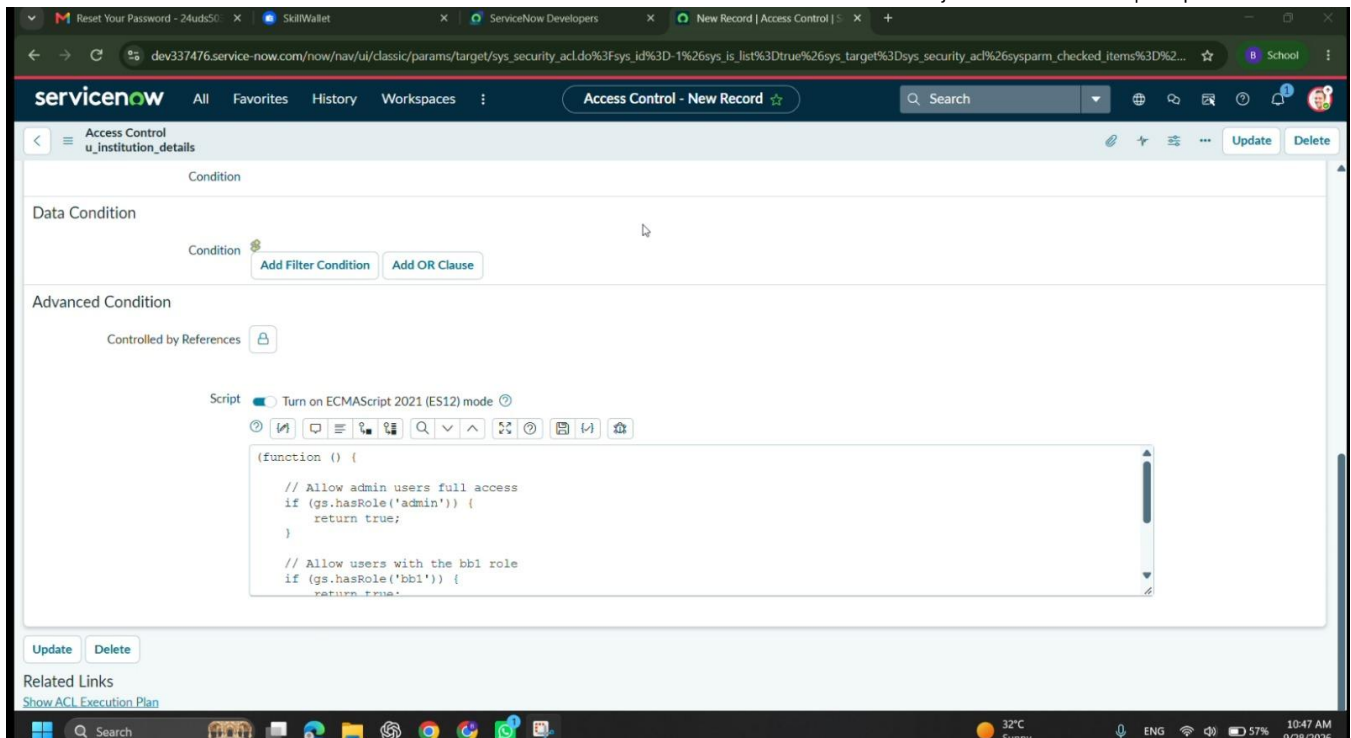


5. Milestone 1 — Create Users and Roles

The source project begins by creating a test user named EEE User and then creating custom roles. The role bb1 is used in the Read ACL, while bb2, bb3, and bb4 are used for Create, Write, and Delete access.

5.1 Create the Test User

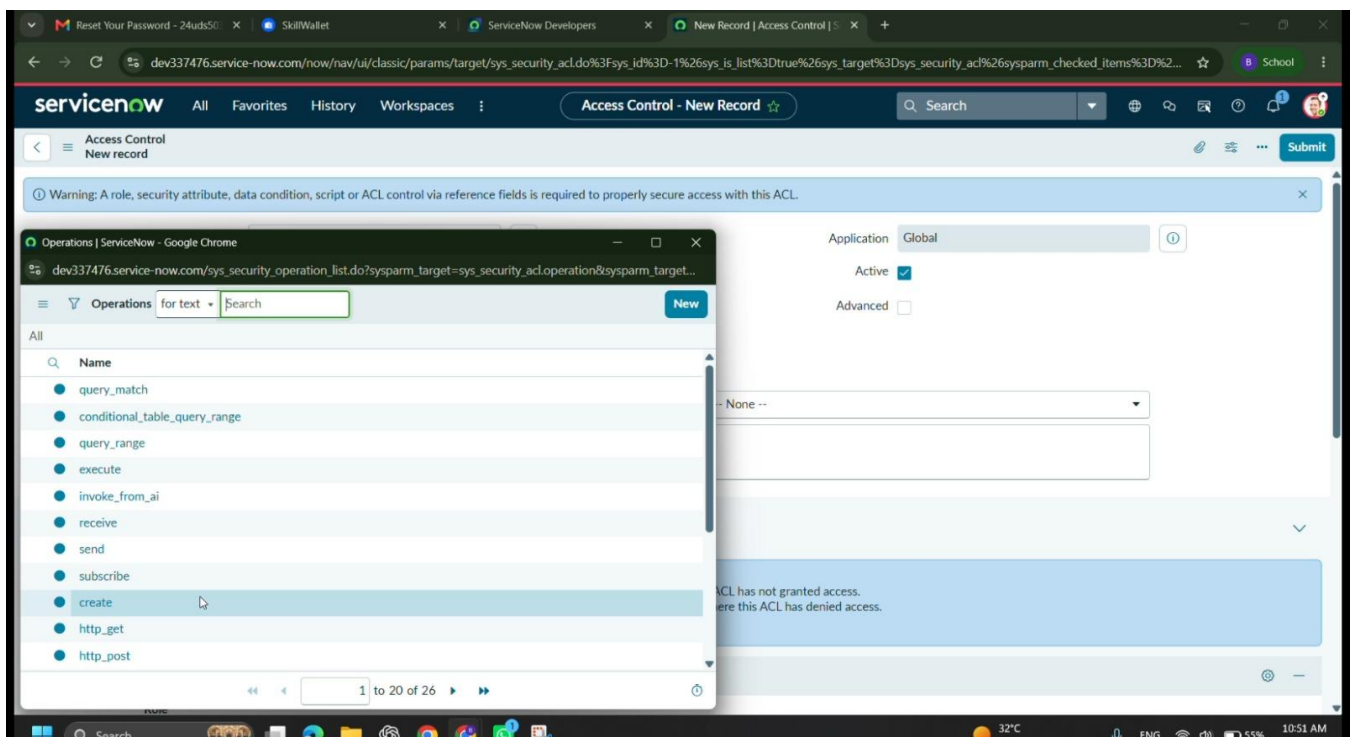
7. Log in to the ServiceNow instance with administrator access.
8. Navigate to User Administration → Users.
9. Click New.
10. Create the test user using the project values: User ID = EEE User, First name = EEE, Last name = User.
11. Enter the required email and save the user.



5.2 Create Custom Roles

12. Navigate to User Administration → Roles.
13. Click New.
14. Create the bb1 role.
15. Repeat the same procedure for bb2, bb3, and bb4.
16. Assign the required roles to the test user for the corresponding validation scenarios.

The source material specifically records bb1 as the custom role required for the Read ACL and states that bb2, bb3, and bb4 are created for the later ACL operations. [filecite2turn0file12L23-L302](#)



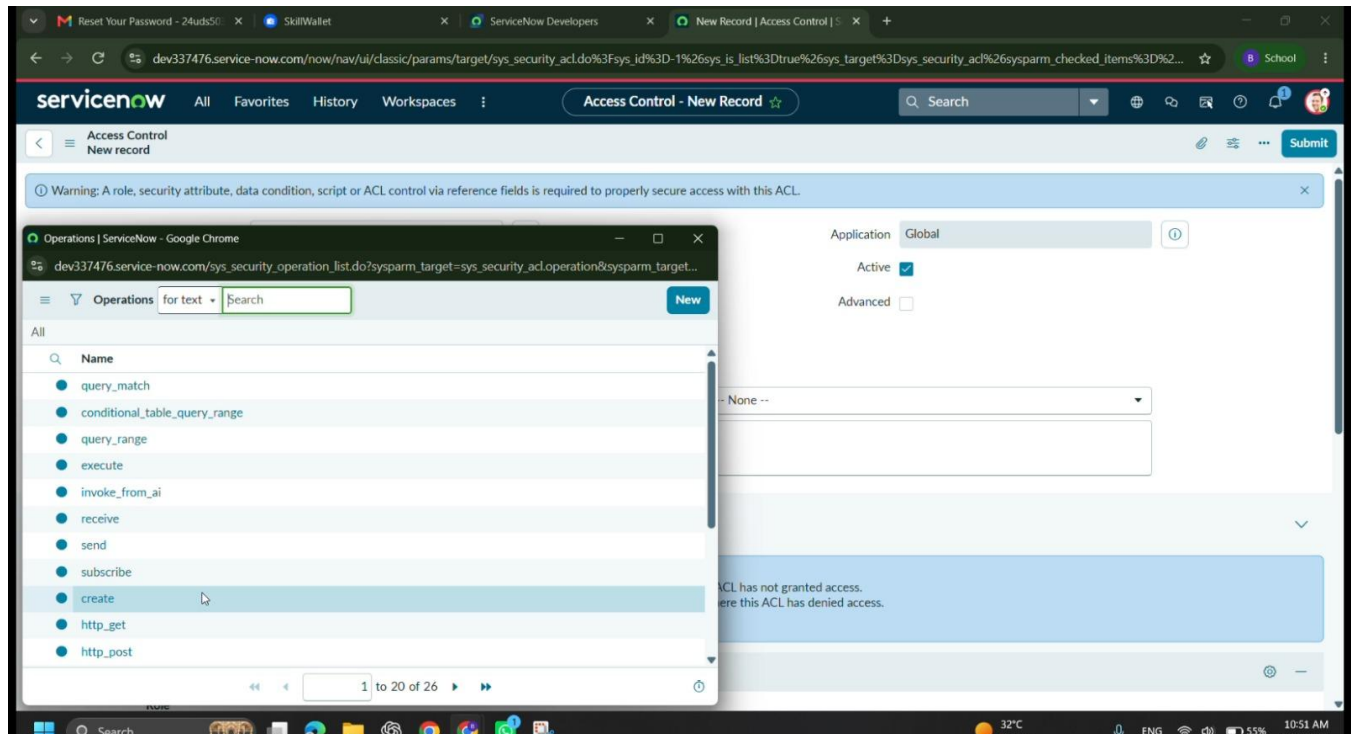
6. Milestone 2 — Create the Institution Details Table

A custom table named Institution Details is created to hold the records used by the ACL project. The technical table name is u_institution_details.

Field	Type	Purpose
Student Roll Number	Auto Number	Unique student record identifier
Student Name	Reference — User	Links the student to a ServiceNow user
Faculty Name	Reference — User	Links the faculty member to a ServiceNow user
Branch	Choice	ECE, EEE, CSE
Email	String	Student/faculty contact email
Phone Number	String	Contact number
Description	Multi String	Additional record information

Configuration Steps

17. In the Application Navigator, search for System Definition → Tables.
18. Click New.
19. Set Label to Institution Details.
20. Set Name to u_institution_details.
21. Leave Extends for as false, as specified by the source project.
22. Submit the table.
23. Add the fields listed above and save the table.
24. Create multiple records using different Branch values: ECE, EEE, and CSE.



7. Milestone 3 — Create the Script-Controlled READ ACL

The Read ACL is the core security component of the project. It is configured as a record-level Read ACL on `u_institution_details`, marked Active and Advanced, with `bb1` in the Requires role list and Branch = `EEE` as the Data condition. The supplied script then performs the role-based authorization decision.

Setting	Configured Value
Type	record
Operation	read
Name	<code>u_institution_details</code>
Active	true
Advanced	true
Requires role	<code>bb1</code>
Data condition	Branch is <code>EEE</code>
Script	Admin → allow; <code>bb1</code> → allow; all others → deny

Configuration Steps

25. Navigate to System Security → Access Control (ACL).
26. Elevate the `security_admin` role.
27. Click New.
28. Set Type to record.
29. Set Operation to read.
30. Set Name to `u_institution_details`.
31. Set Active to true.
32. Enable Advanced.
33. In Requires role, add `bb1`.
34. Add the Data condition: Branch is `EEE`.
35. Enter the supplied ACL script.
36. Click Submit to save the ACL.

Read ACL Script

```
(function () {
  // Allow admin users full access
  if (gs.hasRole('admin')) {
    return true;
  }

  // Allow only EEE branch users to see EEE records
  if (gs.hasRole('bb1')) {
    return true;
  }

  // Deny access for all others
  return false;
})();
```

The script itself checks roles; the supplied project configuration separately specifies Branch = `EEE` as the Read ACL data condition. This distinction is important when explaining the implementation: role authorization and record-field filtering are configured together in the ACL. [filecite2turn0file1L65-L83](#)

8. Script Logic Explanation

The script is intentionally small and uses a clear allow/deny structure.

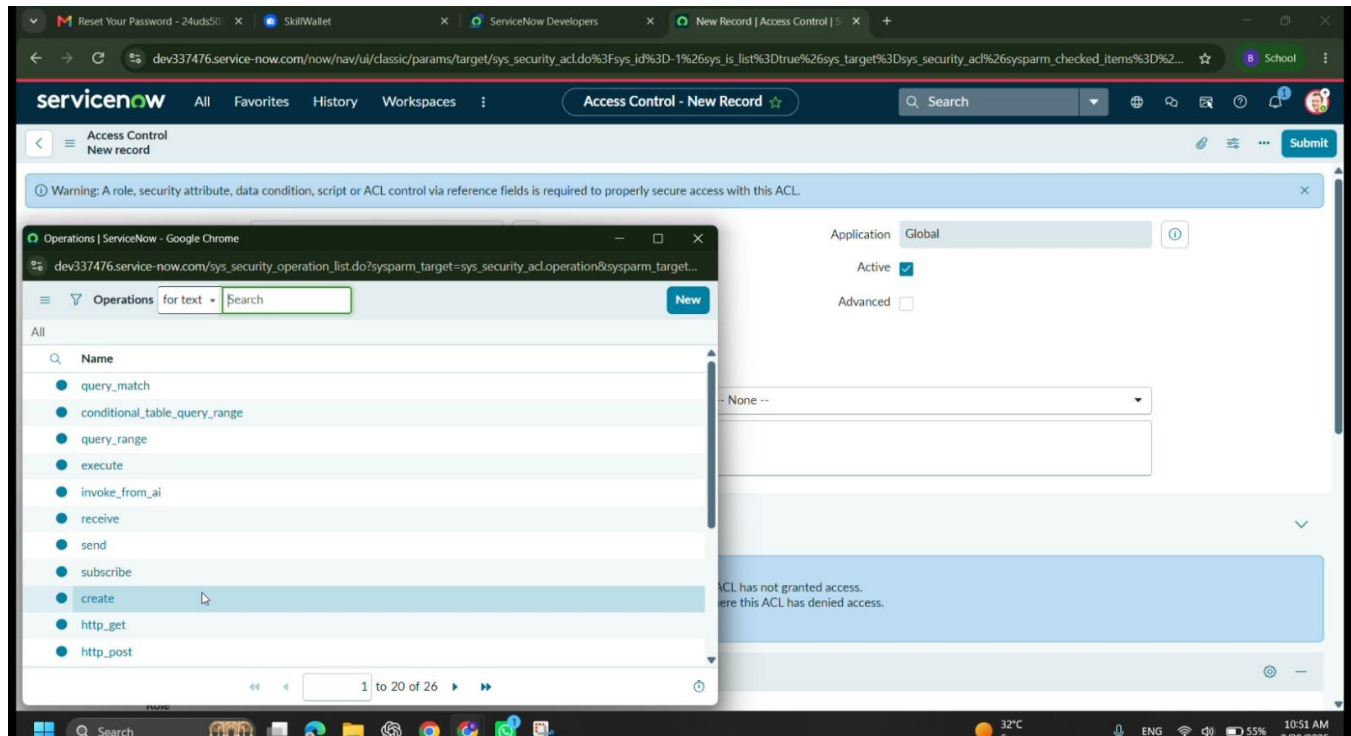
Code Section	Meaning
if (gs.hasRole('admin'))	Checks whether the current user has the ServiceNow admin role.
return true;	Immediately grants access when the admin condition is met.
if (gs.hasRole('bb1'))	Checks whether the current user has the custom bb1 role.
return true;	Grants access when the bb1 role condition is met.
return false;	Denies access to users who do not satisfy the earlier conditions.

Important Project Interpretation

The project documentation describes bb1 as the EEE access role and uses Branch = EEE as the Read ACL data condition. Therefore, when presenting the project, describe the control as a combination of a role requirement and a branch-based record condition rather than saying that the JavaScript alone checks the Branch field.

Why an ACL Script Is Used

- It provides explicit authorization logic in the ACL.
- The allow conditions are easy to read and demonstrate.
- The final false return creates a clear default-deny behavior for users who fail the checks.
- It complements the ACL's role and data-condition configuration.



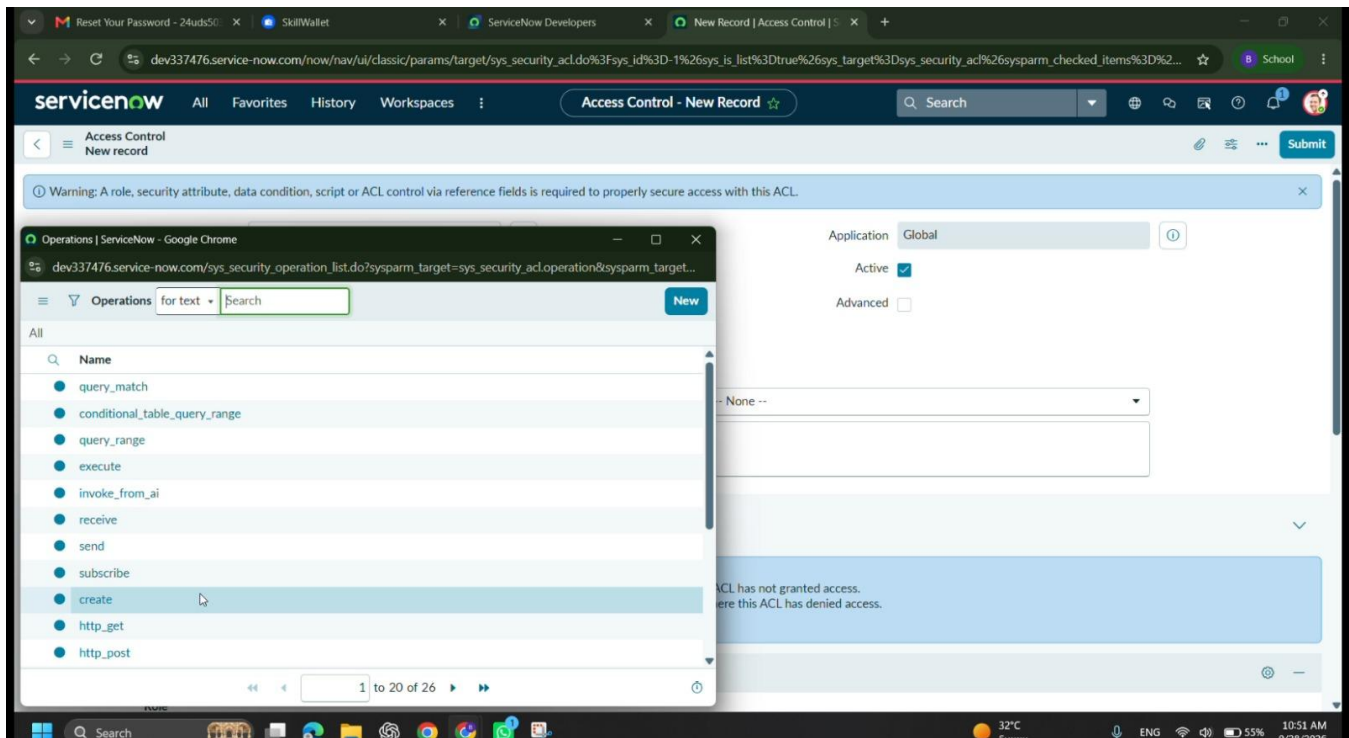
9. Milestone 4 — READ ACL Testing & Verification

The source project validates the Read ACL by impersonating different user types and opening the Institution Details list. The expected outcomes demonstrate that the access rules are applied at record level.

Test User	Role / Condition	Expected Result
EEE User	bb1 + Read ACL condition	Can view EEE branch records
User without bb1	No required Read role	Cannot view records through the configured Read ACL
Admin	admin role	Can view all records regardless of branch

Verification Procedure

37. Impersonate the EEE user.
38. Open the Institution Details list using the table list path.
39. Verify the records visible to the user.
40. Impersonate a user without the required role and repeat the test.
41. Impersonate an administrator and repeat the test.
42. Capture screenshots for each result.



10. Milestone 5 — CREATE ACL

The Create ACL controls whether a user can create new records in `u_institution_details`. The project assigns the `bb2` role to this operation and does not add a data condition.

Setting	Configured Value
Type	record
Operation	create
Name	<code>u_institution_details</code>
Active	true
Requires role	<code>bb2</code>
Data condition	None

Configuration Steps

43. Open Access Control (ACL) and click New.
44. Set Type to record.
45. Set Operation to create.
46. Set Name to `u_institution_details`.
47. Set Active to true.
48. Add `bb2` in Requires role.
49. Leave the Data condition empty.
50. Save the ACL record.
51. Test whether the authorized user can see and use the New button.

The screenshot displays the ServiceNow interface for configuring the 'Institution Details' table. The 'Dictionary Entries' section lists various fields with their types, references, and default values. The 'Description' field is currently selected, showing its type as 'String'. The interface includes standard navigation and action buttons at the top and bottom.

Column label	Type	Reference	Max length	Default value	Display
Updated by	String	(empty)	40	40	false
Updates	Integer	(empty)	40	40	false
Updated	Date/Time	(empty)	40	40	false
Sys ID	Sys ID (GUID)	(empty)	32	32	false
Created by	String	(empty)	40	40	false
Created	Date/Time	(empty)	40	40	false
Student Roll Number	String				false
Student Name	Reference	User			false
Faculty Name	Reference	User			false
Branch	Choice				false
Email	String				false
Phone Number	String				false
Description	String				false

11. Milestone 6 — WRITE ACL

The Write ACL controls whether a user can edit existing records. The source project assigns bb3 to the Write ACL and does not use a data condition for this operation.

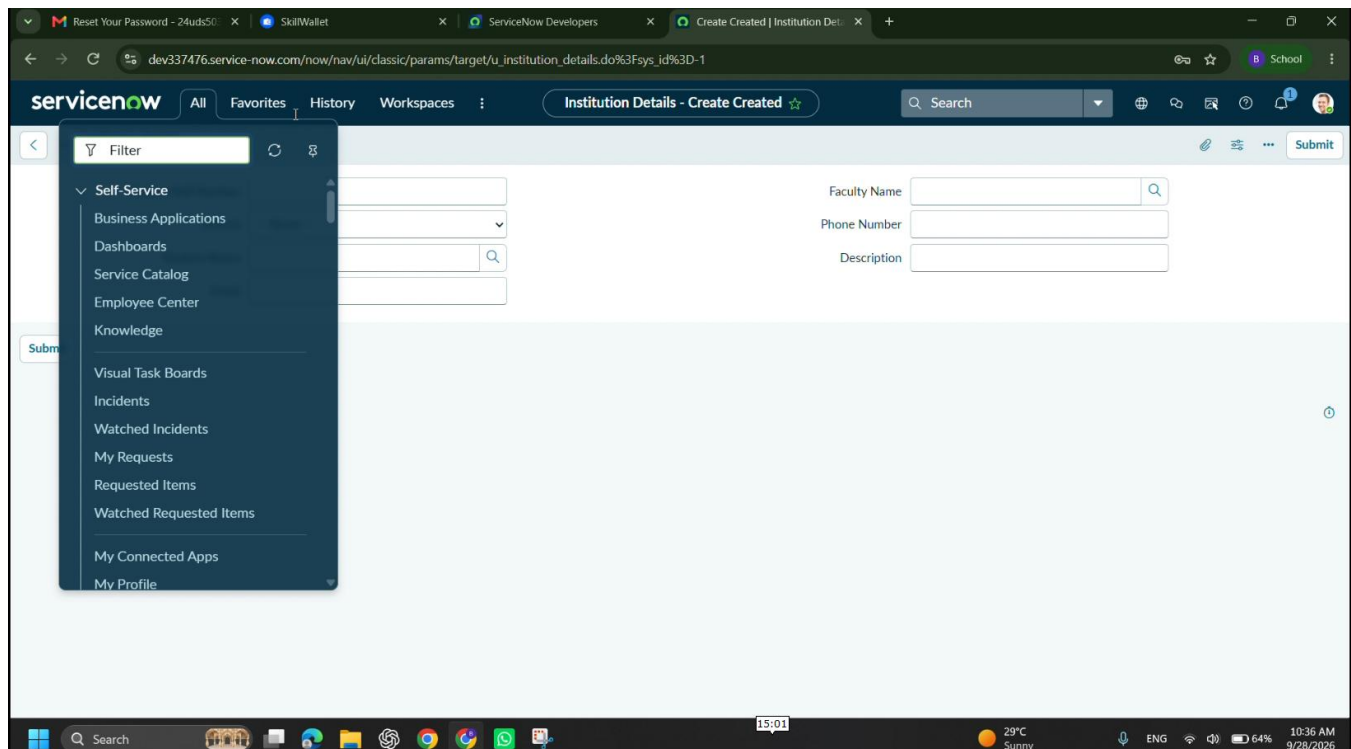
Setting	Configured Value
Type	record
Operation	write
Name	u_institution_details
Active	true
Requires role	bb3
Data condition	None

Configuration Steps

52. Click New to create a Write ACL.
53. Set Type to record.
54. Set Operation to Write.
55. Set Name to u_institution_details.
56. Set Active to true.
57. Add bb3 in Requires role.
58. Leave Data condition empty.
59. Save the ACL.
60. Test editing an existing EEE record with the appropriate role combination.

The source verification states that users holding bb1, bb2, and bb3 can view and edit the EEE records and use the New button. This demonstrates how multiple operation-specific ACLs combine to provide the intended capabilities.

[filecite2turn0file12L119-L1292](#)



12. Milestone 7 — DELETE ACL

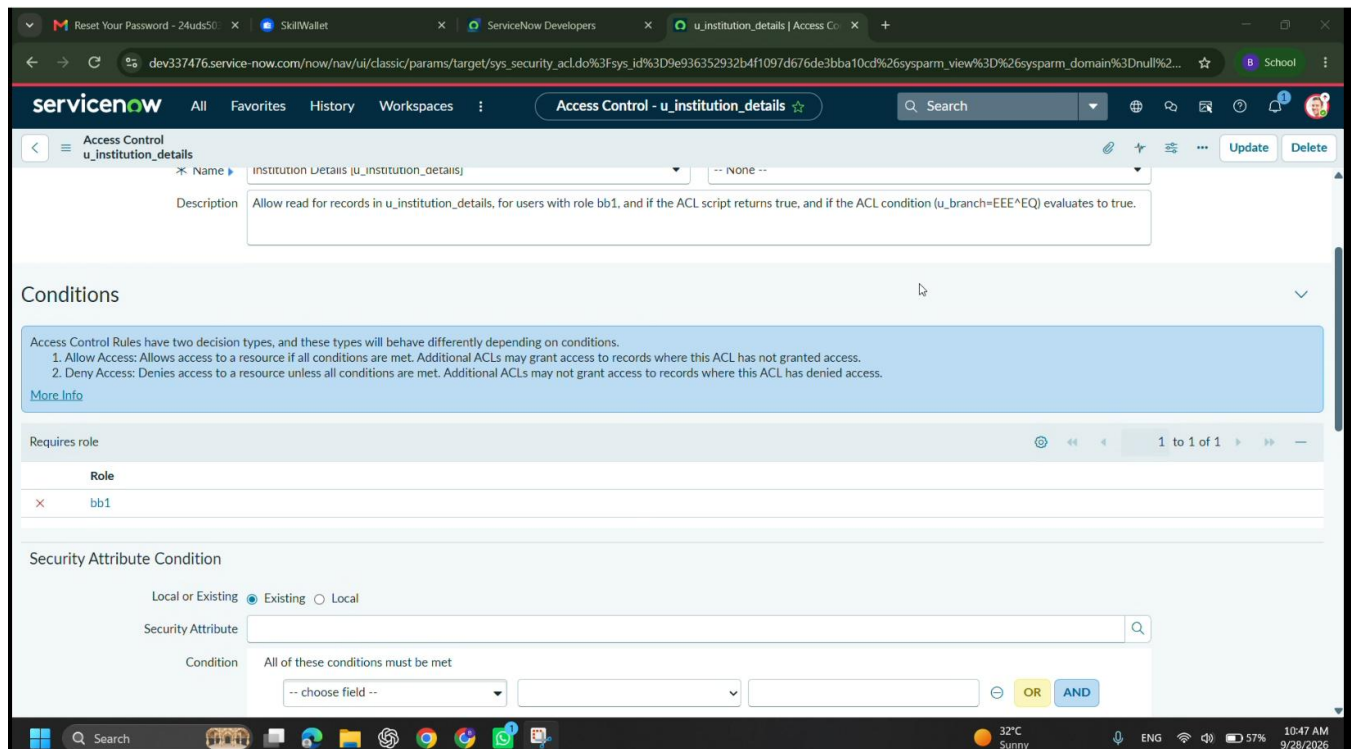
The Delete ACL controls whether a user can remove existing records. The source project assigns bb4 to this operation and does not use a data condition.

Setting	Configured Value
Type	record
Operation	delete
Name	u_institution_details
Active	true
Requires role	bb4
Data condition	None

Configuration Steps

61. Click New to create a Delete ACL.
62. Set Type to record.
63. Set Operation to Delete.
64. Set Name to u_institution_details.
65. Set Active to true.
66. Add bb4 in Requires role.
67. Leave Data condition empty.
68. Save the ACL.
69. Test deletion using the appropriate role combination.

The source project verifies that users with bb1, bb2, bb3, and bb4 can see, create, edit, and delete the EEE branch records. The final project outcome is described as demonstrating READ, WRITE, CREATE, and DELETE ACL enforcement. [filecite2turn0file12L144-L155](#)



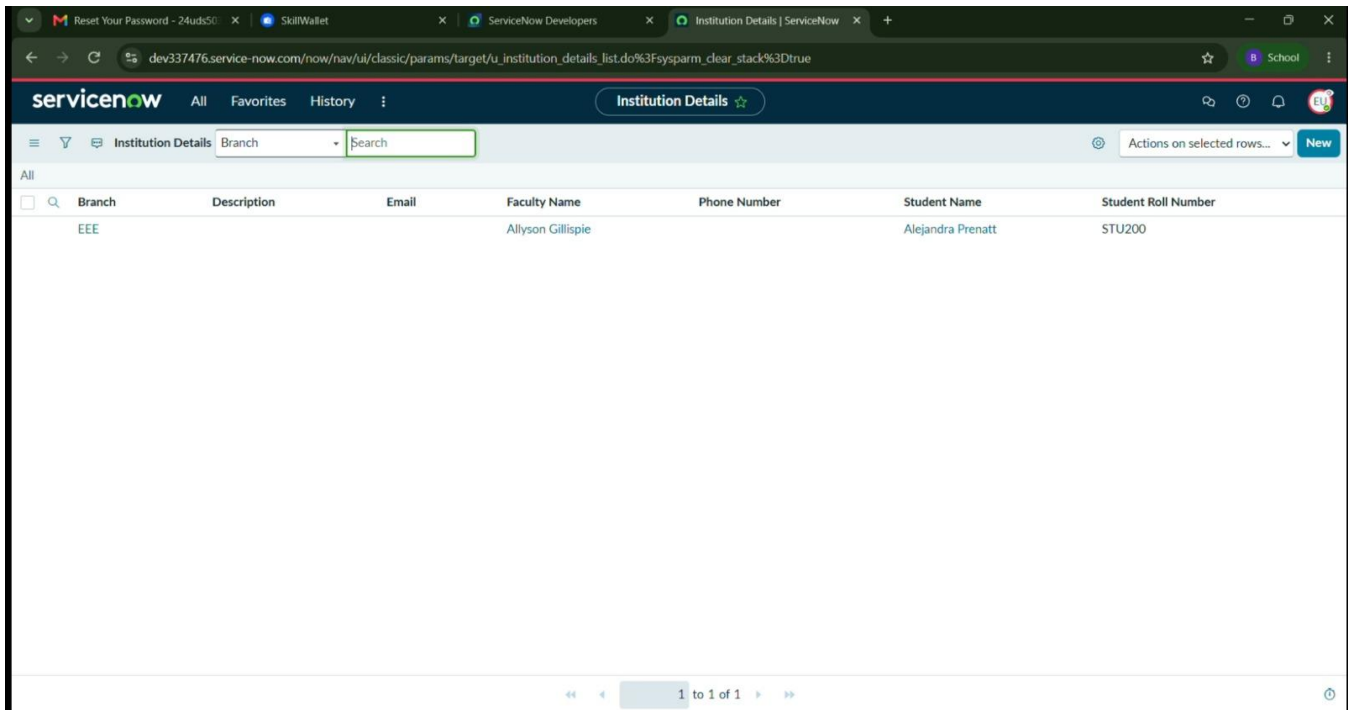
13. End-to-End Testing & Validation

Testing should be performed using impersonation so that each ACL decision is observed from the perspective of the relevant user. Record the actual result in the project report and attach the corresponding screenshot.

Test ID	Scenario	Expected Result	Screenshot
T01	bb1 user opens Institution Details	EEE records are visible according to the Read ACL configuration	
T02	User without required role opens table	Access is restricted by the configured ACL	
T03	Admin opens Institution Details	Admin can view all records	
T04	Authorized create user	New record creation is available	
T05	Unauthorized create user	Create operation is restricted	
T06	Authorized write user	Existing record can be edited	
T07	Unauthorized write user	Edit operation is restricted	
T08	Authorized delete user	Delete operation is available	
T09	Unauthorized delete user	Delete operation is restricted	

Recommended Evidence

- ACL configuration screenshot.
- ACL script screenshot.
- Role assignment screenshot.
- Table and field configuration screenshot.
- EEE records screenshot.
- Unauthorized access screenshot.
- Administrator access screenshot.
- Create / Write / Delete verification screenshots.

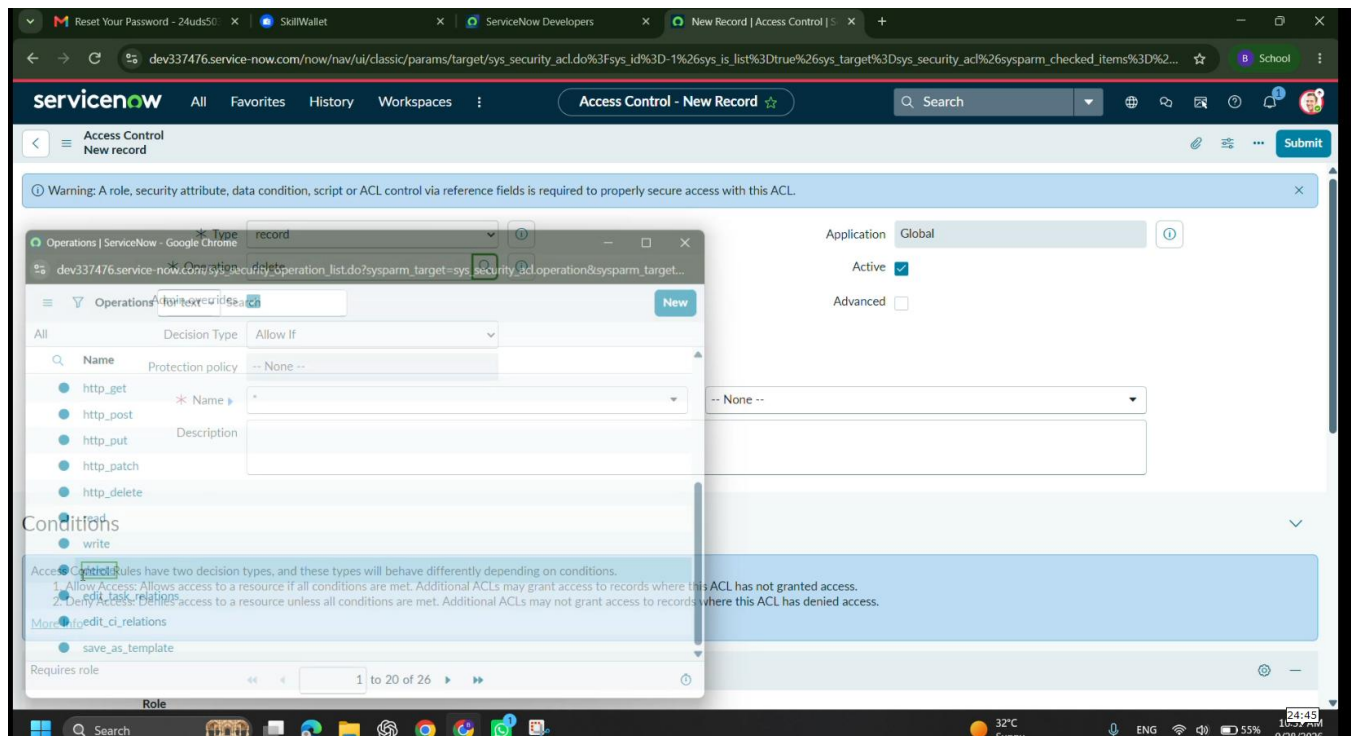


14. Troubleshooting & Validation Checklist

Problem	What to Check
ACL does not appear to work	Confirm ACL is Active and the target table/operation are correct.
Role-based access fails	Verify the required custom role is assigned to the test user.
Read access is unexpected	Check the Branch = EEE data condition and the ACL script.
Cannot edit ACL	Elevate the security_admin role before modifying ACL records.
New button is missing	Check the Create ACL and whether the required role is assigned.
Edit is blocked	Check the Write ACL and bb3 role assignment.
Delete is blocked	Check the Delete ACL and bb4 role assignment.
Test results are confusing	Use impersonation and test one role combination at a time.

Final Checklist

- ☐ EEE User created.
- ☐ bb1, bb2, bb3, bb4 roles created.
- ☐ Institution Details table created.
- ☐ Required fields added.
- ☐ ECE, EEE, CSE records created.
- ☐ Read ACL configured.
- ☐ Branch = EEE condition configured.
- ☐ ACL script added.
- ☐ Create ACL configured.
- ☐ Write ACL configured.
- ☐ Delete ACL configured.
- ☐ All impersonation tests completed.
- ☐ Screenshots inserted into this documentation.



15. Conclusion

The Script-Controlled ACL project demonstrates how ServiceNow can enforce record-level security by combining custom roles, record data conditions, and scripted authorization logic. The implementation begins with a controlled user and role model, creates a dedicated Institution Details table, and then applies operation-specific ACLs for Read, Create, Write, and Delete.

The key Read ACL uses Branch = EEE as the record condition and the supplied script to allow administrators and bb1 users while denying other users. The later ACLs extend the security model by assigning bb2 to Create, bb3 to Write, and bb4 to Delete. The project therefore provides a practical demonstration of layered ServiceNow authorization.

According to the supplied project material, the final outcome is an understanding of how script-controlled READ, WRITE, CREATE, and DELETE ACLs evaluate roles and record field values to protect data from unauthorized viewing, modification, creation, and deletion. [filecite2turn0file12L149-L1552](#)

Project Evidence Index

Evidence No.	Screenshot to Insert	Status
E01	User creation	
E02	Role creation / assignment	
E03	Table creation	
E04	Table fields	
E05	Sample records	
E06	Read ACL + script	
E07	Read ACL — authorized user	
E08	Read ACL — unauthorized user	
E09	Read ACL — admin	
E10	Create ACL	
E11	Write ACL	
E12	Delete ACL	
E13	Final test result	

END OF PROJECT DOCUMENTATION